

Python 3

03	1	<pre> NumberIn = int(input('Enter a positive whole number: ')) NumberOut = 0 Count = 0 while NumberIn > 0: Count += 1 PartValue = NumberIn % 2 NumberIn = NumberIn // 2 for i in range(1, Count): PartValue = PartValue * 10 NumberOut = NumberOut + PartValue print('The result is: ', NumberOut) </pre>	11
14	1	<pre> def DisplayErrorCode(ErrorNumber): print('Error Code ', ErrorNumber, ' - ', end='') if ErrorNumber == 1: print('not a valid piece') elif ErrorNumber == 2: print('not a valid move') elif ErrorNumber == 3: print('not a number') elif ErrorNumber == 4: print('file error') </pre>	3
15	1	<pre> def ValidJump(Board, PlayersPieces, Piece, NewRow, NewColumn): Valid = False MiddlePiece = '' Player = Piece[0].lower() Index = int(Piece[1:]) if Player == 'a': OppositePiecePlayer = 'b' else: OppositePiecePlayer = 'a' if NewRow in range(BOARD_SIZE) and NewColumn in range(BOARD_SIZE): if Board[NewRow][NewColumn] == SPACE: CurrentRow = PlayersPieces[Index][ROW] CurrentColumn = PlayersPieces[Index][COLUMN] MiddlePieceRow = (CurrentRow + NewRow) // 2 MiddlePieceColumn = (CurrentColumn + NewColumn) // 2 MiddlePiece = Board[MiddlePieceRow][MiddlePieceColumn] MiddlePiecePlayer = MiddlePiece[0].lower() if MiddlePiecePlayer == OppositePiecePlayer: Valid = True return Valid Alternative logic statement: MiddlePiecePlayer == OppositePiecePlayer and MiddlePiecePlayer != ' ': </pre>	2
16	1	<pre> def CountNumberOfPieces(PlayersPieces): Count = 0 for Index in range(1, NUMBER_OF_PIECES + 1): if PlayersPieces[Index][ROW] > -1: </pre>	9